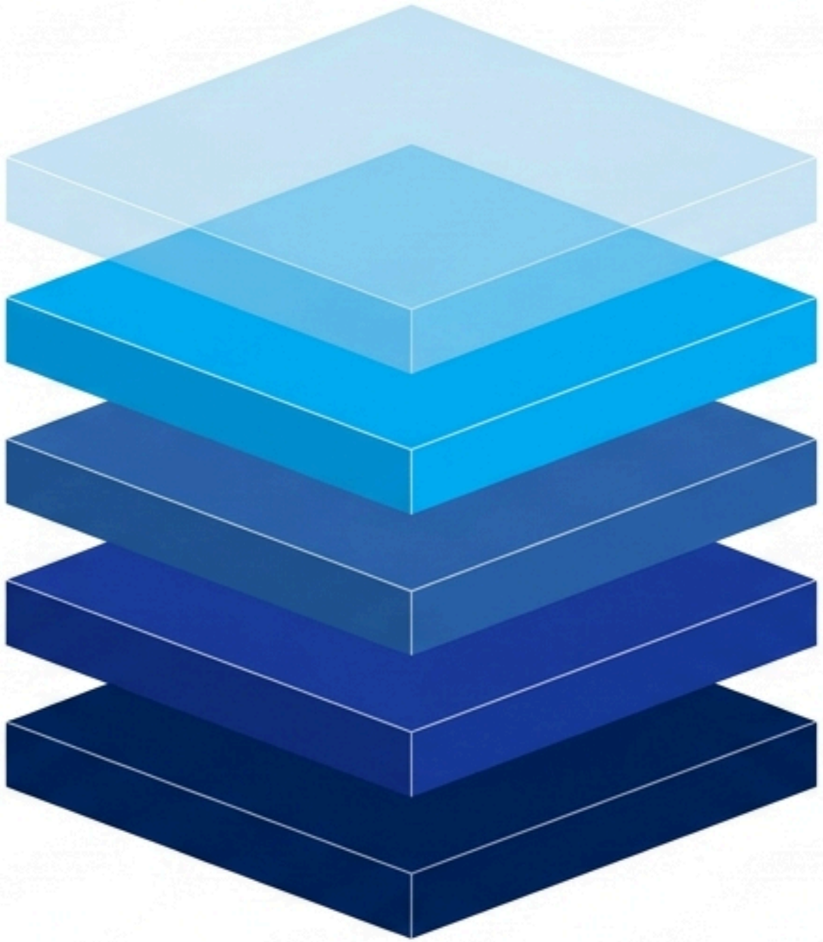
Ormedian
Research Institute



Which Python Ran My Code?

Research Computing Environment

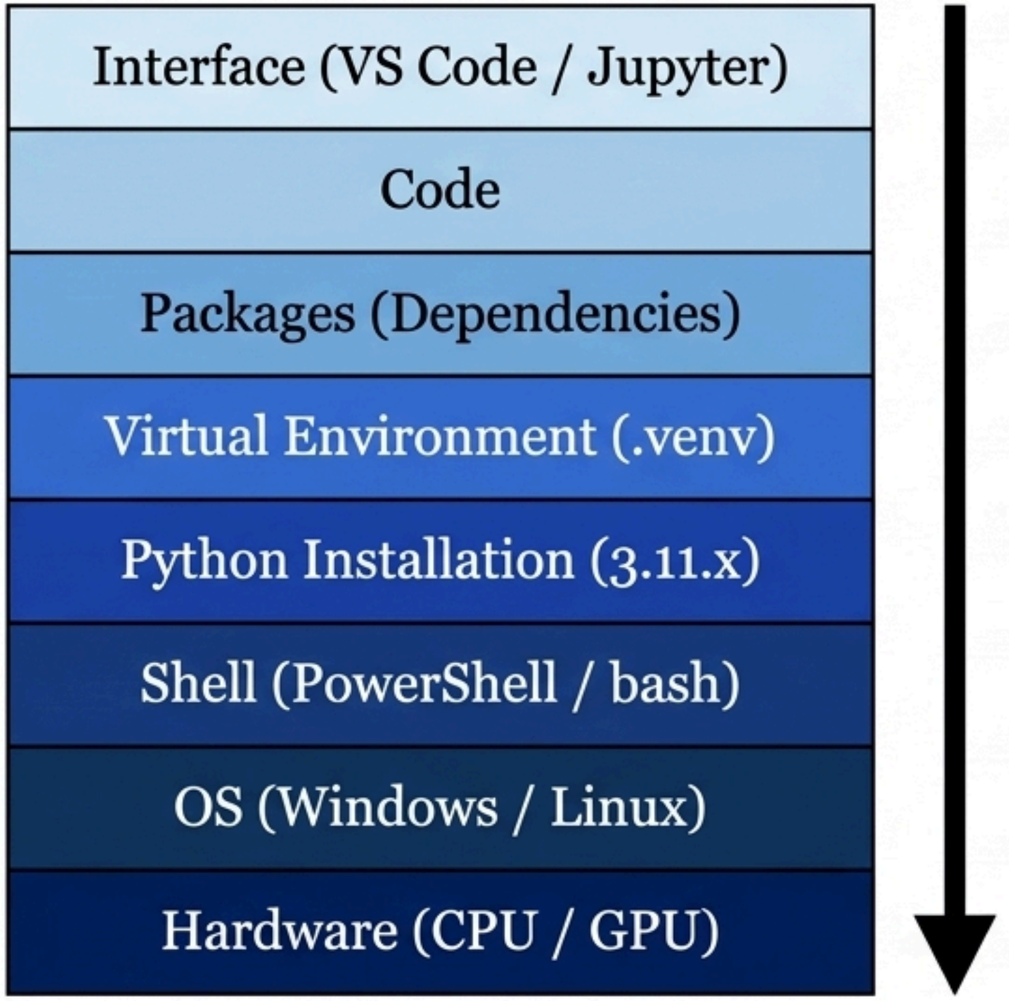
Author: Daniel Oyewale | Unit: P0-U02

The Environment is Part of the Experiment

	Run A	Run B
Code Commit	Same	Same
Seed	42	42
Python & Packages	?	?
AUROC	**0.82**	**0.80**

Key Insight: Until the environment is recorded, the performance gap is not evidence about the model—it is an uncontrolled variable.

Moving Down the Stack



Move down the stack from evidence, not guesswork.

A research computing environment is larger than just Python or the editor. To troubleshoot, systematically identify the exact layer where execution intent separates from system reality.

Assumption vs. Evidence

The Test (Command)	The Observation (Assumption)	The Reality (Evidence)
`py -0p`	Shows 3.10 default, Anaconda 3.9, no 3.11	Only shows which Pythons the launcher knows.
`where.exe python`	Shows Python310 first, WindowsApps second	Only shows the PATH search order.
`sys.executable`	Shows `...\Python310\python.exe`	**Definitive Evidence:** Shows exactly what process ran the code.

Only `sys.executable`, printed from inside the running process, proves execution. The rest are merely candidates.

The Virtual Environment Boundary

Shell Activation (The Convenience)

Action
`.venv\Scripts\Activate.ps1`

Mechanism
Temporarily prepends `.venv\Scripts` to the current shell's PATH.

Limitation
Only applies to this specific terminal session.

Direct Invocation (The Guarantee)

Action
`.venv\Scripts\python.exe`

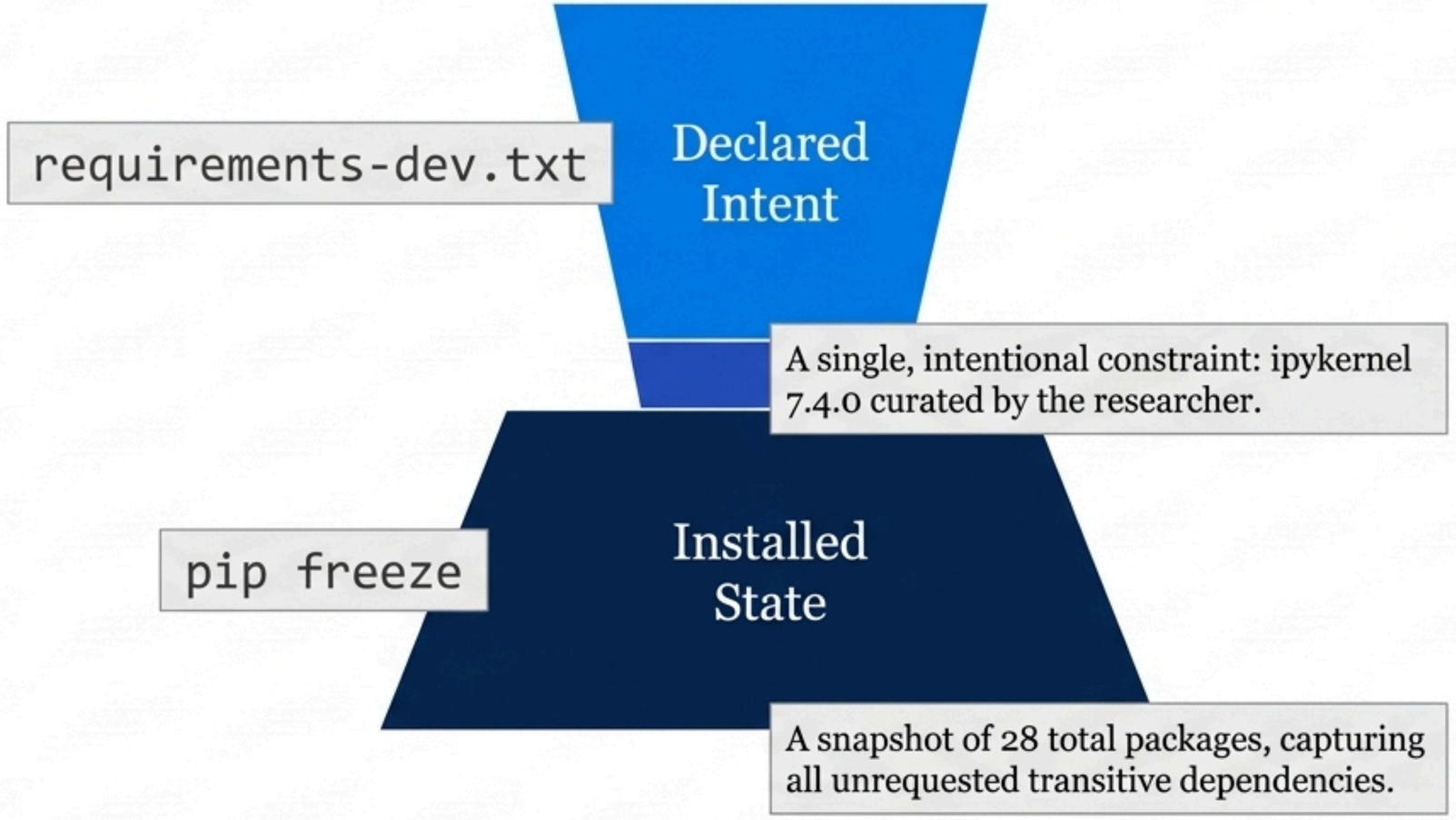
Mechanism
Bypasses the shell PATH entirely. The interpreter inherently finds its own packages via pyvenv.cfg.

Advantage
Works universally across scripts, debuggers, and isolated processes.

```
sys.prefix != sys.base_prefix
pip 24.0 in .venv\Lib\site-packages
```

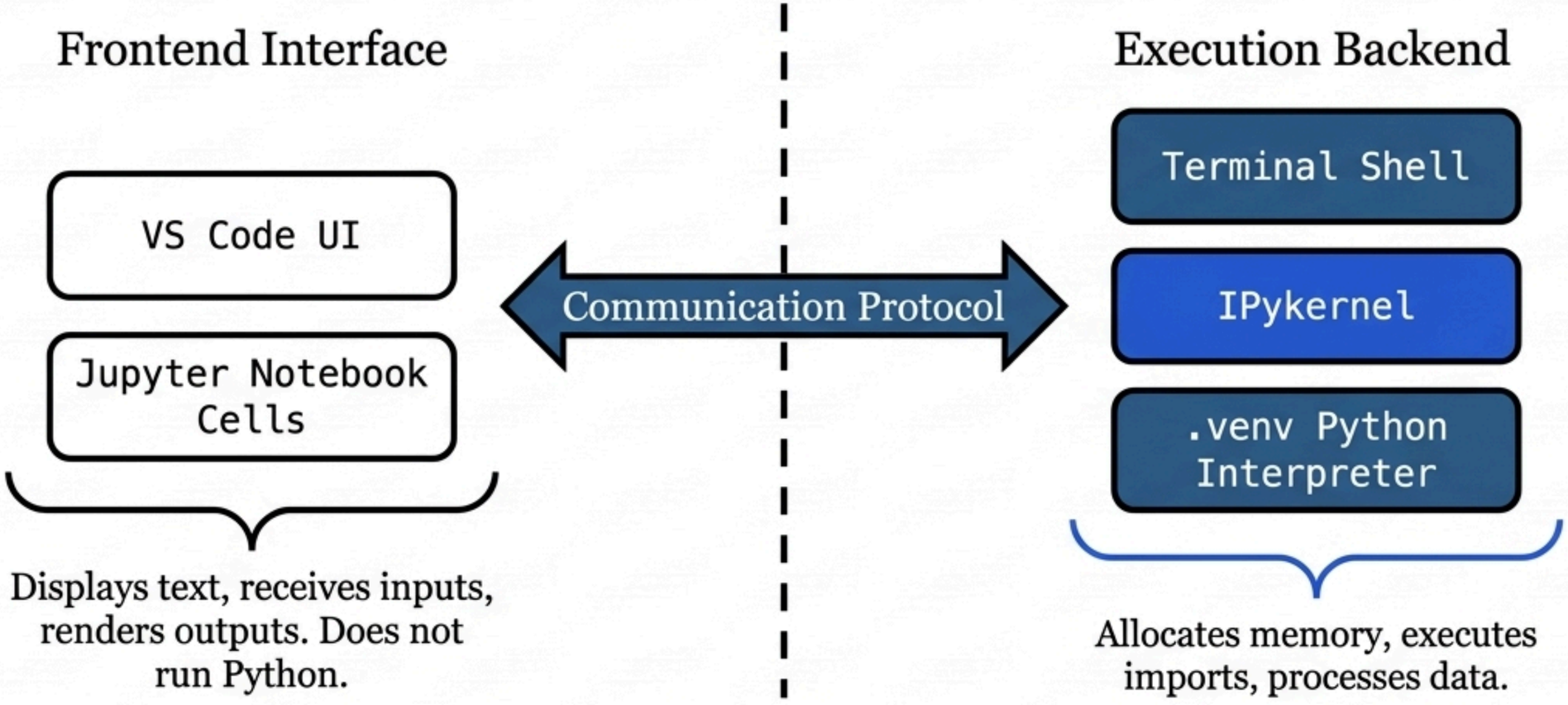
Inspection Lens

Dependency Recording: Intent vs. State



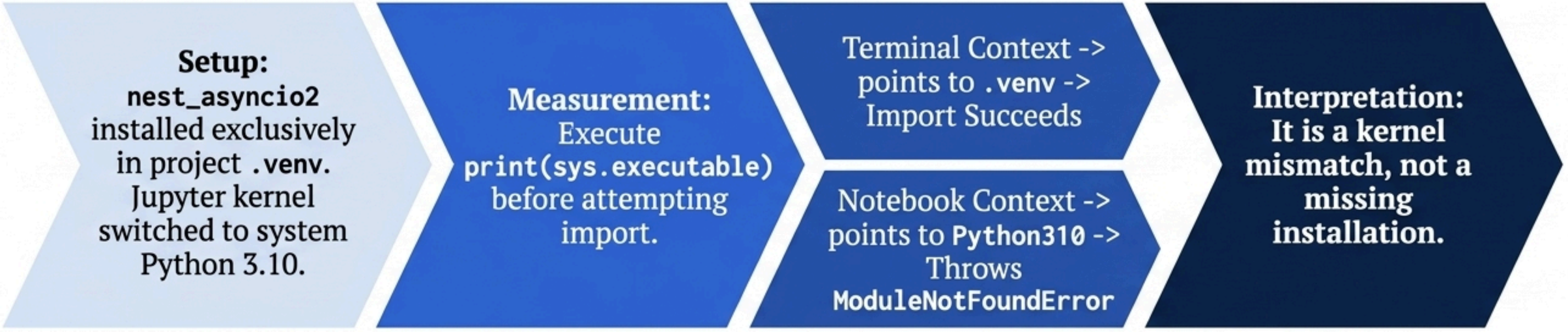
A dependency declaration communicates what the project needs. An environment snapshot records what happened to be installed. Treating a freeze as intent obscures the architecture.

Disentangling the Interface

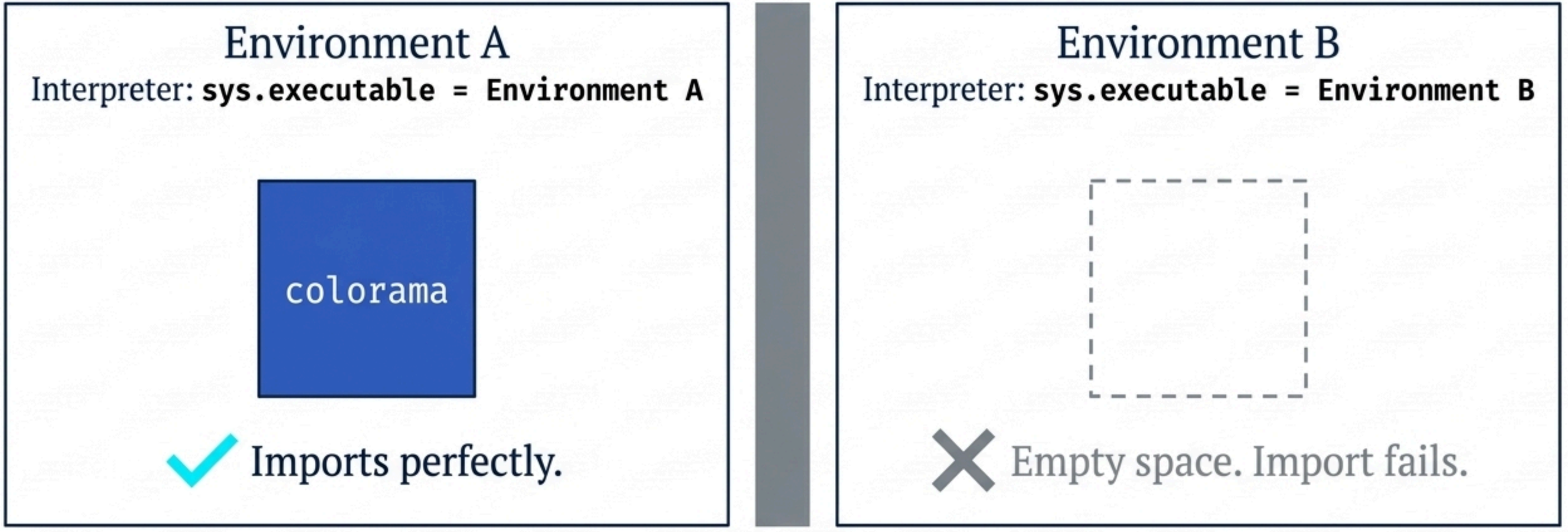


The interface status bar can lie. The execution backend is the sole source of truth for the environment.

Experiment: The Terminal/Notebook Mismatch



Failure Mode: “But it’s already installed…”



The Diagnostic Fix: Never run a bare `pip show`. Always prepend the exact failing interpreter to query the correct context.

```
python -m pip show colorama
```

Ensuring Code Portability

Not This (Fragile)

```
data_path = 'C:\Users\Daniel\Ormedian\data\scans'
```

⚠

Silently depends on a single developer's local machine layout. Instantly breaks for collaborators.

Do This (Portable)

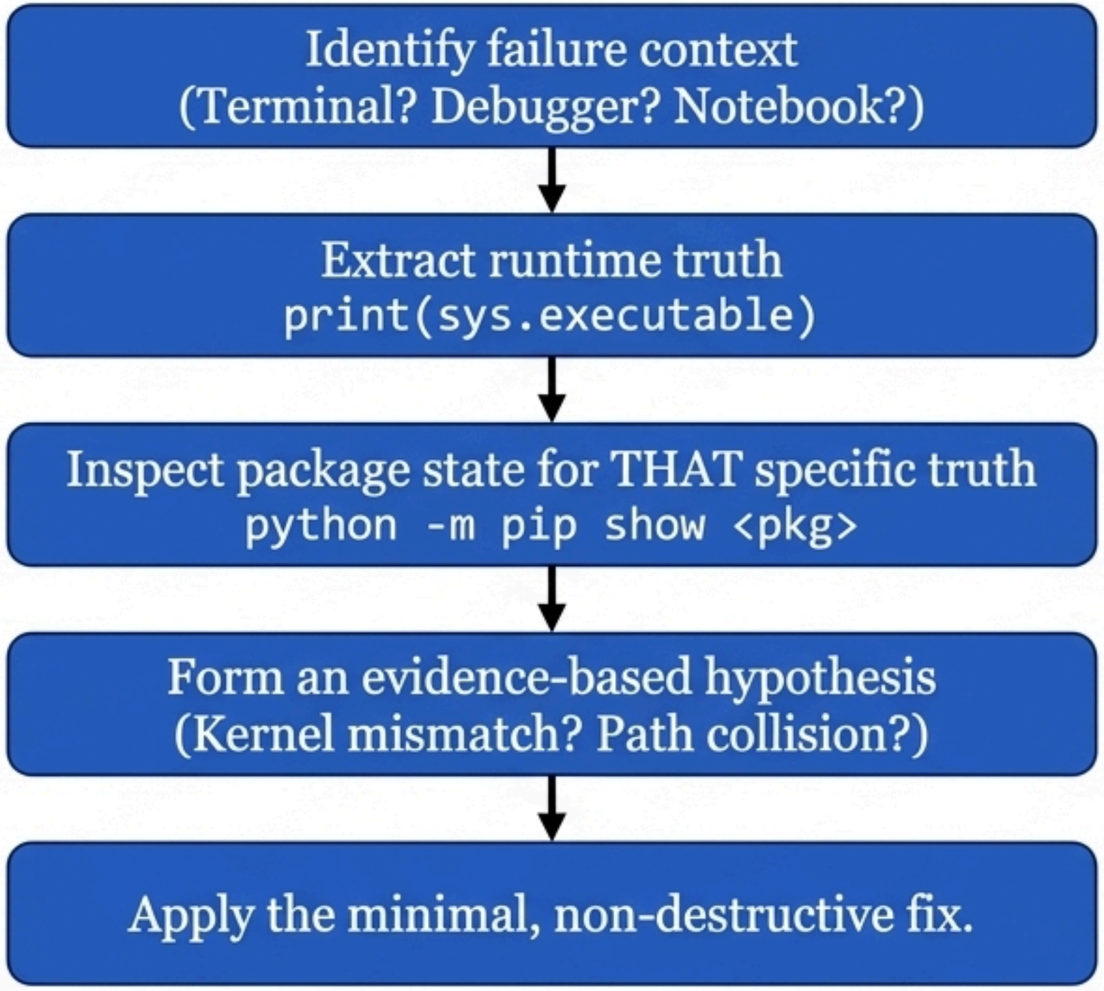
```
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
data_path = PROJECT_ROOT / 'data' / 'scans'
```

Explicit, project-relative pathing that dynamically resolves regardless of the host machine.

Takeaway: Reproducible research code assumes nothing about the host directory structure.

The Troubleshooting Decision Tree



Core Philosophy: Never guess. Never blindly reinstall.
Always extract evidence from the failing context first.

Moving Forward: What I'll Do Differently

1. Print `sys.executable` inside the failing context before altering any configurations.
2. Always use `python -m pip` to ensure installations land exactly where the executing Python runs.
3. Record the code commit, explicit interpreter path, and a `pip freeze` snapshot with every experimental result.
4. Treat environment differences as the primary hypothesis before treating outcome differences as scientific findings.

A reliable researcher doesn't just make it work on their machine—they make it reproducible, inspectable, and portable.